

machines on the local network. We return to RPC semantics in Section 8.3 when discussing RPC in the presence of failures.

4.3 Message-oriented communication

Remote procedure calls and remote object invocations contribute to hiding communication in distributed systems, that is, they enhance access transparency. Unfortunately, neither mechanism is always appropriate. In particular, when it cannot be assumed that the receiving side is executing at the time a request is issued, alternative communication services are needed. Likewise, the inherent synchronous nature of RPCs, by which a client is blocked until its request has been processed, may need to be replaced by something else.

That something else is messaging. In this section we concentrate on message-oriented communication in distributed systems by first taking a closer look at what exactly synchronous behavior is and what its implications are. Then, we discuss messaging systems that assume that parties are executing at the time of communication. Finally, we will examine message-queuing systems that allow processes to exchange information, even if the other party is not executing at the time communication is initiated.

Simple transient messaging with sockets

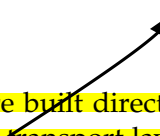
Many distributed systems and applications are built directly on top of the simple message-oriented model offered by the transport layer. To better understand and appreciate the message-oriented systems as part of middleware solutions, we first discuss messaging through transport-level sockets.

Special attention has been paid to standardizing the interface of the transport layer to allow programmers to make use of its entire suite of (messaging) protocols through a simple set of operations. Also, standard interfaces make it easier to port an application to a different machine. As an example, we briefly discuss the **socket interface** as introduced in the 1970s in Berkeley Unix, and which has been adopted as a POSIX standard (with only very few adaptations).

Conceptually, a **socket** is a communication end point to which an application can write data that are to be sent out over the underlying network, and from which incoming data can be read. A socket forms an abstraction over the actual port that is used by the local operating system for a specific transport protocol. In the following text, we concentrate on the socket operations for TCP, which are shown in Figure 4.18.

Servers generally execute the first four operations, normally in the order given. When calling the socket operation, the caller creates a new communication end point for a specific transport protocol. Internally, creating a communication end point means that the local operating system reserves resources for sending and receiving messages for the specified protocol.

Camada de Transporte -
protocolos TCP e UDP



Operation	Description
socket	Create a new communication end point
bind	Attach a local address to a socket
listen	Tell operating system what the maximum number of pending connection requests should be
accept	Block caller until a connection request arrives
connect	Actively attempt to establish a connection
send	Send some data over the connection
receive	Receive some data over the connection
close	Release the connection

Figure 4.18: The socket operations for TCP/IP.

The **bind** operation associates a local address with the newly created socket. For example, a server should bind the IP address of its machine together with a (possibly well-known) port number to a socket. Binding tells the operating system that the server wants to receive messages only on the specified address and port. In the case of connection-oriented communication, the address is used to receive incoming connection requests.

The **listen** operation is called only in the case of connection-oriented communication. It is a nonblocking call that allows the local operating system to reserve enough buffers for a specified maximum number of pending connection requests that the caller is willing to accept.

A call to **accept** blocks the caller until a connection request arrives. When a request arrives, the local operating system creates a new socket with the same properties as the original one, and returns it to the caller. This approach will allow the server to, for example, fork off a process that will subsequently handle the actual communication through the new connection. The server can go back and wait for another connection request on the original socket.

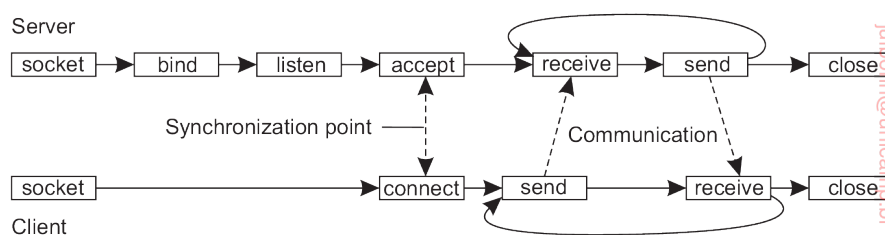


Figure 4.19: Connection-oriented communication pattern using sockets.

Let us now take a look at the client side. Here, too, a socket must first be created using the **socket** operation, but explicitly binding the socket to a local address is not necessary, since the operating system can dynamically allocate a port when the connection is set up. The **connect** operation requires that the

caller specifies the transport-level address to which a connection request is to be sent. The client is blocked until a connection has been set up successfully, after which both sides can start exchanging information through the send and receive operations. Finally, closing a connection is symmetric when using sockets, and is established by having both the client and server call the close operation. Although there are many exceptions to the rule, the general pattern followed by a client and server for connection-oriented communication using sockets is as shown in Figure 4.19. Details on network programming using sockets and other interfaces in Unix can be found in [Stevens, 1998].

Note 4.7 (Example: A simple socket-based client-server system)

As an illustration of the recurring pattern in Figure 4.19, consider the simple socket-based client-server system shown below (see also Note 2.1). We see the server [Figure 4.20(a)] starting by creating a socket, and subsequently binding an address to that socket. It calls the listen operation, and waits for an incoming connection request. When the server accepts a connection, the socket library creates a separate connection, `conn`, which is used to receive data and send a response to the connected client. The server enters a loop receiving and sending messages, until no more data has been received. It then closes the connection.

```

1 from socket import *
2 s = socket(AF_INET, SOCK_STREAM)
3 s.bind((HOST, PORT))
4 s.listen(1)
5 (conn, addr) = s.accept() # returns new socket and addr. client
6 while True:               # forever
7     data = conn.recv(1024) # receive data from client
8     if not data: break     # stop if client stopped
9     conn.send(str(data)+"*") # return sent data plus an "*"
10 conn.close()             # close the connection

```

Figure 4.20: (a) A simple socket-based client-server system: the server.

The client, shown in Figure 4.20(b), again follows the pattern from Figure 4.19. It creates a socket, and calls connect to request a connection with the server. Once the connection has been established, it sends a single message, waits for the response, and after printing the result, closes the connection.

```

1 from socket import *
2 s = socket(AF_INET, SOCK_STREAM)
3 s.connect((HOST, PORT)) # connect to server (block until accepted)
4 s.send('Hello, world') # send same data
5 data = s.recv(1024)    # receive the response
6 print data             # print the result
7 s.close()              # close the connection

```

Figure 4.20: (b) A simple socket-based client-server system: the client.

Note 4.8 (Advanced: Implementing stubs as global references revisited)

To provide a deeper insight in the working of sockets, let us look at a more elaborate example, namely the use of stubs as global references. We return to our example of implementing a shared list, which we now do by means of a list server, implemented in the form of the Python class shown in Figure 4.21(b). Figure 4.21(a) shows the stub implementation of a shared list. Again, we have omitted code for readability.

```

1  class DBClient:
2      def sendrecv(self, message):
3          sock = socket()                # create a socket
4          sock.connect((self.host, self.port)) # connect to server
5          sock.send(pickle.dumps(message))    # send some data
6          result = pickle.loads(sock.recv(1024)) # receive the response
7          sock.close()                     # close the connection
8          return result
9
10     def create(self):
11         self.listID = self.sendrecv([CREATE])
12         return self.listID
13
14     def getValue(self):
15         return self.sendrecv([GETVALUE, self.listID])
16
17     def appendData(self, data):
18         return self.sendrecv([APPEND, data, self.listID])

```

Figure 4.21: (a) Implementing a list server in Python: the list structure.

The DBClient class represents a client-side stub that, once marshaled, can be passed between processes. It provides three operations associated with a list: create, getValue, and append, with obvious semantics. A DBClient is assumed to be associated with one specific list as managed by the server. An identifier for that list is returned when the list is created. Note how the (internal) sendrecv operation follows the client-side pattern explained in Figure 4.19.

The server maintains lists, as shown in Figure 4.21(b). Its internal data structure is a setOfLists with each element being a previously created list. The server simply waits for incoming requests, unmarshals the request, and checks which operation is being requested. Results are sent back to the requesting client (which always issues the sendrecv operation implemented as part of DBClient). Again, we see that the server follows the pattern shown in Figure 4.19: it creates a socket, binds an address to it, informs the operating system to how many connections it should listen, and then waits to accept an incoming connection request. Once a connection has been established, the server receives data, sends a response, and closes the connection again.

```

1 class Server:
2     def __init__(self, port=PORT):
3         self.host = 'localhost'           # this machine
4         self.port = port                   # the port it will listen to
5         self.sock = socket()              # socket for incoming calls
6         self.sock.bind((self.host,self.port)) # bind socket to an address
7         self.sock.listen(5)                # max num of connections
8         self.setOfLists = {}              # init: no lists to manage
9
10    def run(self):
11        while True:
12            (conn, addr) = self.sock.accept() # accept incoming call
13            data = conn.recv(1024)           # fetch data from client
14            request = pickle.loads(data)     # unwrap the request
15            if request[0] == CREATE:         # create a list
16                listID = len(self.setOfLists) + 1 # allocate listID
17                self.setOfLists[listID] = []    # initialize to empty
18                conn.send(pickle.dumps(listID)) # return ID
19
20            elif request[0] == APPEND:       # append request
21                listID = request[2]          # fetch listID
22                data = request[1]           # fetch data to append
23                self.setOfLists[listID].append(data) # append it to the list
24                conn.send(pickle.dumps(OK))  # return an OK
25
26            elif request[0] == GETVALUE:    # read request
27                listID = request[1]         # fetch listID
28                result = self.setOfLists[listID] # get the elements
29                conn.send(pickle.dumps(result)) # return the list
30            conn.close()                    # close the connection

```

Figure 4.21: (b) Implementing a list server in Python: the server.

To use a stub as a global reference, we represent each client application by means of the class `Client` shown in Figure 4.21(c). The class is instantiated in the same process running the application (exemplified by the value of `self.host`), and will be listening on a specific port for messages from other applications, as well as the server. Otherwise, it merely sends and receives messages, coded through the operations `sendTo` and `recvAny`, respectively.

Now consider the code shown in Figure 4.21(d), which mimics two client applications. The first one creates a new list and appends data to it. Then note how `dbClient1` is simply sent to the other client. Under the hood, we now know that it is marshaled in the operation `sendTo` (line 12) of class `Client` shown in Figure 4.21(c).

```

1 class Client:
2     def __init__(self, port):
3         self.host = 'localhost'           # this machine
4         self.port = port                  # port it will listen to
5         self.sock = socket()              # socket for incoming calls
6         self.sock.bind((self.host, self.port)) # bind socket to an address
7         self.sock.listen(2)                # max num connections
8
9     def sendTo(self, host, port, data):
10        sock = socket()
11        sock.connect((host, port))         # connect to server (blocking call)
12        sock.send(pickle.dumps(data))      # send some data
13        sock.close()
14
15    def recvAny(self):
16        (conn, addr) = self.sock.accept()
17        return conn.recv(1024)

```

Figure 4.21: (c) Implementing a list server in Python: the client.

```

1 pid = os.fork()
2 if pid == 0:
3     client1 = Client(CLIENT1)           # create client
4     dbClient1 = DBClient(HOST,PORT)     # create reference
5     dbClient1.create()                  # create new list
6     dbClient1.appendData('Client 1')    # append some data
7     client1.sendTo(HOSTCL2,CLIENT2,dbClient1) # send to other client
8
9 pid = os.fork()
10 if pid == 0:
11     client2 = Client(CLIENT2)           # create a new client
12     data = client2.recvAny()             # block until data is sent
13     dbClient2 = pickle.loads(data)       # receive reference
14     dbClient2.appendData('Client 2')     # append data to same list

```

Figure 4.21: (d) Passing stubs as references.

The second client simply waits for an incoming message (line 12), unmarshals the result, knowing that it is a `DBClient` instance, and subsequently appends some more data to the same list as the one the first client appended data. Indeed, an instance of `DBClient` is seen to be passed as global reference, seemingly along with all the operations that go with the associated class.

Advanced transient messaging

The standard socket-based approach toward transient messaging is very basic and as such, rather brittle: a mistake is easily made. Furthermore, sockets essentially support only TCP or UDP, meaning that any extra facility for

messaging needs to be implemented separately by an application programmer. In practice, we do often need more advanced approaches for message-oriented communication to make network programming easier, to expand beyond the functionality offered by existing networking protocols, to make better use of local resources, and so on.

Using messaging patterns: ZeroMQ

One approach toward making network programming easier is based on the observation that many messaging applications, or their components, can be effectively organized according to a few simple communication patterns. By subsequently providing enhancements to sockets for each of these patterns, it may become easier to develop a networked, distributed application. This approach has been followed in ZeroMQ and documented in [Hintjens, 2013; Akgul, 2013].

Like in the Berkeley approach, ZeroMQ also provides sockets through which all communication takes place. Actual message transmission generally takes place over TCP connections, and like TCP, all communication is essentially connection-oriented, meaning that a connection will first be set up between a sender and receiver before message transmission can take place. However, setting up, and maintaining connections is kept mostly under the hood: an application programmer need not bother with those issues. To further simplify matters, a socket may be bound to multiple addresses, effectively allowing a server to handle messages from very different sources through a single interface. For example, a server can listen to multiple ports using a single blocking receive operation. ZeroMQ sockets can thus support *many-to-one* communication instead of just *one-to-one* communication as is the case with standard Berkeley sockets. To complete the story: ZeroMQ sockets also support *one-to-many* communication, i.e., multicasting.

Essential to ZeroMQ is that communication is asynchronous: a sender will normally continue after having submitted a message to the underlying communication subsystem. An interesting side effect of combining asynchronous with connection-oriented communication, is that a process can request a connection setup, and subsequently send a message even if the recipient is not yet up-and-running and ready to accept incoming connection requests, let alone incoming messages. What happens, of course, is that a connection request and subsequent messages are queued at the sender's side, while a separate thread as part of ZeroMQ's library will take care that eventually the connection is set up and messages are transmitted to the recipient.

Simplifying matters, ZeroMQ establishes a higher level of abstraction in socket-based communication by *pairing* sockets: a specific type of socket used for sending messages is paired with a corresponding socket type for receiving messages. Each pair of socket types corresponds to a communication pattern.

The three most important communication patterns supported by ZeroMQ are *request-reply*, *publish-subscribe*, and *pipeline*.

The **request-reply pattern** is used in traditional client-server communication, like the ones normally used for remote procedure calls. A client application uses a **request socket** (of type REQ) to send a request message to a server and expects the latter to respond with an appropriate response. The server is assumed to use a **reply socket** (of type REP). The request-reply pattern simplifies matters for developers by avoiding the need to call the listen operation, as well as the accept operation. Moreover, when a server receives a message, a subsequent call to send is automatically targeted toward the original sender. Likewise, when a client calls the recv operation (for receiving a message) after having sent a message, ZeroMQ assumes the client is waiting for a response from the original recipient. Note that this approach was effectively encoded in the local sendrecv operation of Figure 4.21 discussed in Note 4.8.

Note 4.9 (Example: The request-reply pattern)

Let us look at a simple programming example to illustrate the request-reply pattern. Figure 4.22(a) shows a server that appends an asterisk to a received message. As before, it creates a socket, and binds it to a combination of a protocol (in this case TCP), and a host and port. In our example, the server is willing to accept incoming connection requests on two different ports. It then waits for incoming messages. The request-reply pattern effectively ties the receipt of a message to the subsequent response. In other words, when the server calls send, it will transmit a message to the same client from which it previously had received a message. Of course, this simplicity can be achieved only if the programmer indeed abides to the request-reply pattern.

```

1  import zmq
2  context = zmq.Context()
3
4  p1 = "tcp://" + HOST + ":" + PORT1 # how and where to connect
5  p2 = "tcp://" + HOST + ":" + PORT2 # how and where to connect
6  s = context.socket(zmq.REP)      # create reply socket
7
8  s.bind(p1)                       # bind socket to address
9  s.bind(p2)                       # bind socket to address
10 while True:
11     message = s.recv()            # wait for incoming message
12     if not "STOP" in message:    # if not to stop...
13         s.send(message + "*")    # append "*" to message
14     else:                        # else...
15         break                   # break out of loop and end

```

Figure 4.22: (a) A ZeroMQ client-server system based: the server.

The client, shown in Figure 4.22(b) does what is expected: it creates a socket and connects to the associated server. When it sends a message, it can expect to

receive, from that same server, a response. By sending the string “STOP”, it tells the server it is done, after which the server will actually stop.

```

1 import zmq
2 context = zmq.Context()
3
4 p1 = "tcp://" + HOST + ":" + PORT1 # how and where to connect
5 s = context.socket(zmq.REQ) # create request socket
6
7 s.connect(p1) # block until connected
8 s.send("Hello world 1") # send message
9 message = s.recv() # block until response
10 s.send("STOP") # tell server to stop
11 print message # print result

```

Figure 4.22: (b) A ZeroMQ client-server system: the client.

Interestingly, the asynchronous nature of ZeroMQ allows one to start the client *before* starting the server. An implication is that if, in this example, we would start the server, then a client, and after a while a second client, that the latter will be blocked until the server is restarted. Furthermore, note that ZeroMQ does not require the programmer to specify how many bytes are expected to be received. Unlike TCP, ZeroMQ uses messages instead of bytestreams as its underlying model of communication.

In the case of a **publish-subscribe pattern**, clients **subscribe** to specific messages that are **published** by servers. We came across this pattern briefly in Chapter 1 when discussing event-based coordination. In effect, only the messages to which the client has subscribed will be transmitted. If a server is publishing messages to which no one has subscribed, these messages will be lost. In its simplest form, this pattern establishes multicasting messages from a server to several clients. The server is assumed to use a socket of type PUB, while each client must use SUB type sockets. Each client socket is connected to the socket of the server. By default, a client subscribes to no specific message. This means that as long as no explicit subscription is provided, a client will not receive a message published by the server.

Note 4.10 (Example: The publish-subscribe pattern)

Again, let us make this pattern more concrete through a simple example. Figure 4.23(a) shows an admittedly naive time server that publishes its current, local time, through a PUB socket. The local time is published every five seconds, for any interested client.

A client is equally simple, as shown in Figure 4.23(b). It first creates a SUB socket which it connects to the corresponding PUB socket of the server. In order to receive the appropriate messages, it needs to subscribe to messages that have TIME as their tag. In our example, a client will simply print the first five messages

received from the server. Note that we can have as many clients as we want: the server's message will be multicast to all subscribers.

```

1 import zmq, time
2
3 context = zmq.Context()
4 s = context.socket(zmq.PUB)      # create a publisher socket
5 p = "tcp://" + HOST + ":" + PORT # how and where to communicate
6 s.bind(p)                       # bind socket to the address
7 while True:
8     time.sleep(5)                # wait every 5 seconds
9     s.send("TIME " + time.asctime()) # publish the current time

```

Figure 4.23: (a) A multicasting socket-based time server

```

1 import zmq
2
3 context = zmq.Context()
4 s = context.socket(zmq.SUB)      # create a subscriber socket
5 p = "tcp://" + HOST + ":" + PORT # how and where to communicate
6 s.connect(p)                   # connect to the server
7 s.setsockopt(zmq.SUBSCRIBE, "TIME") # subscribe to TIME messages
8
9 for i in range(5): # Five iterations
10     time = s.recv() # receive a message
11     print time

```

Figure 4.23: (b) A client for the multicasting socket-based time server.

Finally, the **pipeline pattern** is characterized by the fact that a process wants to **push out** its results, assuming that there are other processes that want to **pull in** those results. The essence of the pipeline pattern is that a pushing process does not really care which other process pulls in its results: the first available one will do just fine. Likewise, any process pulling in results from multiple other processes will do so from the first pushing process making its results available. The intention of the pipeline pattern is thus seen to keep as many processes working as possible, pushing results through a pipeline of processes as quickly as possible.

Note 4.11 (Example: The pipeline pattern)

As our last example, consider the following template for keeping a collection of worker tasks busy. Figure 4.24(a) shows the code for a so-called **farmer task**: a process generating tasks to be picked up by others. In this example, we simulate the task by letting the farmer pick a random number modeling the duration, or load, of the work to be done. This workload is then sent to the push socket, effectively being queued until another process picks it up.

```

1 import zmq, time, pickle, sys, random
2
3 context = zmq.Context()
4 me = str(sys.argv[1])
5 s = context.socket(zmq.PUSH)      # create a push socket
6 src = SRC1 if me == '1' else SRC2 # check task source host
7 prt = PORT1 if me == '1' else PORT2 # check task source port
8 p = "tcp://" + src + ":" + prt    # how and where to connect
9 s.bind(p)                        # bind socket to address
10
11 for i in range(100):              # generate 100 workloads
12     workload = random.randint(1, 100) # compute workload
13     s.send(pickle.dumps((me, workload))) # send workload to worker

```

Figure 4.24: (a) A task simulating the generation of work.

```

1 import zmq, time, pickle, sys
2
3 context = zmq.Context()
4 me = str(sys.argv[1])
5 r = context.socket(zmq.PULL)      # create a pull socket
6 p1 = "tcp://" + SRC1 + ":" + PORT1 # address first task source
7 p2 = "tcp://" + SRC2 + ":" + PORT2 # address second task source
8 r.connect(p1)                    # connect to task source 1
9 r.connect(p2)                    # connect to task source 2
10
11 while True:
12     work = pickle.loads(r.recv()) # receive work from a source
13     time.sleep(work[1]*0.01)      # pretend to work

```

Figure 4.24: (b) A worker task.

Such other processes are known as **worker tasks**, of which a sketch is given in Figure 4.24(b). A worker task connects a single pull socket to, in this case, two specific farmer tasks. Once it picks up some work, it simulates that it is actually doing something by sleeping for a number of tens of milliseconds proportional to the received workload.

The semantics of this push-pull pattern is such that the first available worker will pick up work from either farmer, and likewise, if there are multiple workers ready to pick up work, each one of them will be provided with a task. How work distribution is actually done in a fair manner requires some specific attention which we will not discuss further here.

The Message-Passing Interface (MPI)

With the advent of high-performance multicomputers, developers have been looking for message-oriented operations that would allow them to easily write highly efficient applications. This means that the operations should be at a

convenient level of abstraction (to ease application development), and that their implementation incurs only minimal overhead. Sockets were deemed insufficient for two reasons. First, they were at the wrong level of abstraction by supporting only simple send and receive operations. Second, sockets had been designed to communicate across networks using general-purpose protocol stacks such as TCP/IP. They were not considered suitable for the proprietary protocols developed for high-speed interconnection networks, such as those used in high-performance server clusters. Those protocols required an interface that could handle more advanced features, such as different forms of buffering and synchronization.

The result was that most interconnection networks and high-performance multicomputers were shipped with proprietary communication libraries. These libraries offered a wealth of high-level and generally efficient communication operations. Of course, all libraries were mutually incompatible, so that application developers now had a portability problem.

The need to be hardware and platform independent eventually lead to the definition of a standard for message passing, simply called the **Message-Passing Interface** or **MPI**. MPI is designed for parallel applications and as such is tailored to transient communication. It makes direct use of the underlying network. Also, it assumes that serious failures such as process crashes or network partitions are fatal and do not require automatic recovery.

MPI assumes communication takes place within a known group of processes. Each group is assigned an identifier. Each process within a group is also assigned a (local) identifier. A (groupID, processID) pair therefore uniquely identifies the source or destination of a message, and is used instead of a transport-level address. There may be several, possibly overlapping groups of processes involved in a computation and that are all executing at the same time.

At the core of MPI are messaging operations to support transient communication, of which the most intuitive ones are summarized in Figure 4.25.

Operation	Description
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until transmission starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

Figure 4.25: Some of the most intuitive message-passing operations of MPI.

Transient asynchronous communication is supported by means of the `MPI_bsend` operation. The sender submits a message for transmission, which is generally first copied to a local buffer in the MPI runtime system. When the message has been copied, the sender continues. The local MPI runtime system will remove the message from its local buffer and take care of transmission as soon as a receiver has called a receive operation.

There is also a blocking send operation, called `MPI_send`, of which the semantics are implementation dependent. The operation `MPI_send` may either block the caller until the specified message has been copied to the MPI runtime system at the sender's side, or until the receiver has initiated a receive operation. Synchronous communication by which the sender blocks until its request is accepted for further processing is available through the `MPI_ssend` operation. Finally, the strongest form of synchronous communication is also supported: when a sender calls `MPI_sendrecv`, it sends a request to the receiver and blocks until the latter returns a reply. Basically, this operation corresponds to a normal RPC.

Both `MPI_send` and `MPI_ssend` have variants that avoid copying messages from user buffers to buffers internal to the local MPI runtime system. These variants essentially correspond to a form of asynchronous communication. With `MPI_isend`, a sender passes a pointer to the message after which the MPI runtime system takes care of communication. The sender immediately continues. To prevent overwriting the message before communication completes, MPI offers operations to check for completion, or even to block if required. As with `MPI_send`, whether the message has actually been transferred to the receiver or that it has merely been copied by the local MPI runtime system to an internal buffer is left unspecified.

Likewise, with `MPI_issend`, a sender also passes only a pointer to the MPI runtime system. When the runtime system indicates it has processed the message, the sender is then guaranteed that the receiver has accepted the message and is now working on it.

The operation `MPI_recv` is called to receive a message; it blocks the caller until a message arrives. There is also an asynchronous variant, called `MPI_irecv`, by which a receiver indicates that it is prepared to accept a message. The receiver can check whether or not a message has indeed arrived, or block until one does.

The semantics of MPI communication operations are not always straightforward, and different operations can sometimes be interchanged without affecting the correctness of a program. The official reason why so many different forms of communication are supported is that it gives implementers of MPI systems enough possibilities for optimizing performance. Cynics might say the committee could not make up its collective mind, so it threw in everything. By now, MPI is in its third version with over 440 operations available. Being designed for high-performance parallel applications, it is perhaps easier

to understand its diversity. More on MPI can be found in [Gropp et al., 2016]. The complete MPI-3 reference can be found in [Message Passing Interface Forum, 2015].

Message-oriented persistent communication

We now come to an important class of message-oriented middleware services, generally known as **message-queuing systems**, or just **message-oriented middleware (MOM)**. Message-queuing systems provide extensive support for persistent asynchronous communication. The essence of these systems is that they offer intermediate-term storage capacity for messages, without requiring either the sender or receiver to be active during message transmission. An important difference with sockets and MPI is that message-queuing systems are typically targeted to support message transfers that are allowed to take minutes instead of seconds or milliseconds.

Message-queuing model

The basic idea behind a message-queuing system is that applications communicate by inserting messages in specific queues. These messages are forwarded over a series of communication servers and are eventually delivered to the destination, even if it was down when the message was sent. In practice, most communication servers are directly connected to each other. In other words, a message is generally transferred directly to a destination server. In principle, each application has its own private queue to which other applications can send messages. A queue can be read only by its associated application, but it is also possible for multiple applications to share a single queue.

An important aspect of message-queuing systems is that a sender is generally given only the guarantees that its message will eventually be inserted in the recipient's queue. No guarantees are given about when, or even if the message will actually be read, which is completely determined by the behavior of the recipient.

These semantics permit communication to be loosely coupled in time. There is thus no need for the receiver to be executing when a message is being sent to its queue. Likewise, there is no need for the sender to be executing at the moment its message is picked up by the receiver. The sender and receiver can execute completely independently of each other. In fact, once a message has been deposited in a queue, it will remain there until it is removed, irrespective of whether its sender or receiver is executing. This gives us four combinations with respect to the execution mode of the sender and receiver, as shown in Figure 4.26.

In Figure 4.26(a), both the sender and receiver execute during the entire transmission of a message. In Figure 4.26(b), only the sender is executing, while the receiver is passive, that is, in a state in which message delivery is not

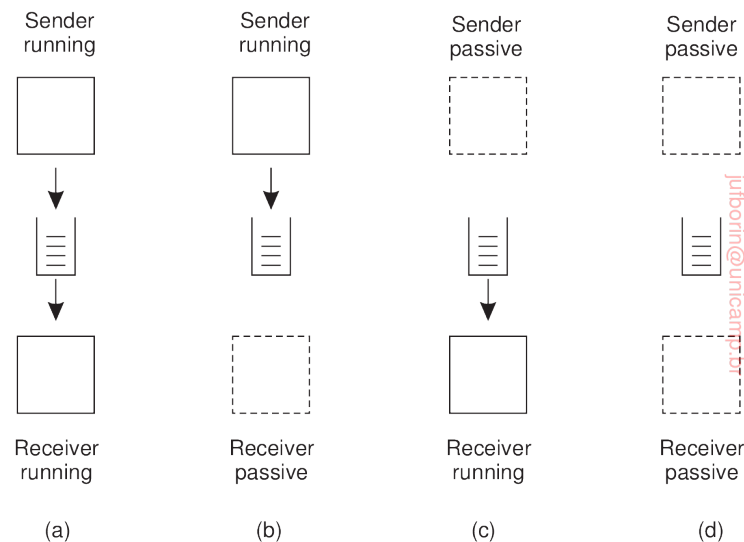


Figure 4.26: Four combinations for loosely-coupled communication using queues.

possible. Nevertheless, the sender can still send messages. The combination of a passive sender and an executing receiver is shown in Figure 4.26(c). In this case, the receiver can read messages that were sent to it, but it is not necessary that their respective senders are executing as well. Finally, in Figure 4.26(d), we see the situation that the system is storing (and possibly transmitting) messages even while sender and receiver are passive. One may argue that only if this last configuration is supported, the message-queuing system truly provides persistent messaging.

Messages can, in principle, contain any data. The only important aspect from the perspective of middleware is that messages are properly addressed. In practice, addressing is done by providing a systemwide unique name of the destination queue. In some cases, message size may be limited, although it is also possible that the underlying system takes care of fragmenting and assembling large messages in a way that is completely transparent to applications. An effect of this approach is that the basic interface offered to applications can be extremely simple, as shown in Figure 4.27.

The put operation is called by a sender to pass a message to the underlying system that is to be appended to the specified queue. As we explained, this is a nonblocking call. The get operation is a blocking call by which an authorized process can remove the longest pending message in the specified queue. The process is blocked only if the queue is empty. Variations on this call allow searching for a specific message in the queue, for example, using a priority, or a matching pattern. The nonblocking variant is given by the poll operation. If the queue is empty, or if a specific message could not be found, the calling

Operation	Description
put	Append a message to a specified queue
get	Block until the specified queue is nonempty, and remove the first message
poll	Check a specified queue for messages, and remove the first. Never block
notify	Install a handler to be called when a message is put into the specified queue

Figure 4.27: Basic interface to a queue in a message-queuing system.

process simply continues.

Finally, most queuing systems also allow a process to install a handler as a *callback function*, which is automatically invoked whenever a message is put into the queue. Callbacks can also be used to automatically start a process that will fetch messages from the queue if no process is currently executing. This approach is often implemented by means of a daemon on the receiver's side that continuously monitors the queue for incoming messages and handles accordingly.

General architecture of a message-queuing system

Let us now take a closer look at what a general message-queuing system looks like. First of all, queues are managed by **queue managers**. A queue manager is either a separate process, or is implemented by means of a library that is linked with an application. Secondly, as a rule of thumb, an application can put messages only into a *local* queue. Likewise, getting a message is possible by extracting it from a *local* queue only. As a consequence, if a queue manager QM_A handling the queues for an application A runs as a separate process, both processes QM_A and A will generally be placed on the same machine, or at worst on the same LAN. Also note that if *all* queue managers are linked into their respective applications, we can no longer speak of a persistent asynchronous messaging system.

If applications can put messages only into local queues, then clearly each message will have to carry information concerning its destination. It is the queue manager's task to make sure that a message reaches its destination. This brings us to a number of issues.

In the first place, we need to consider how the destination queue is addressed. Obviously, to enhance location transparency, it is preferable that queues have logical, location-independent names. Assuming that a queue manager is implemented as a separate process, using logical names implies that each name should be associated with a **contact address**, such as a (host,port)-pair, and that the name-to-address mapping is readily available to a queue

manager, as shown in Figure 4.28. In practice, a contact address carries more information, notably the protocol to be used, such as TCP or UDP. We came across such contact addresses in our examples of advanced sockets in, for example, Note 4.9.

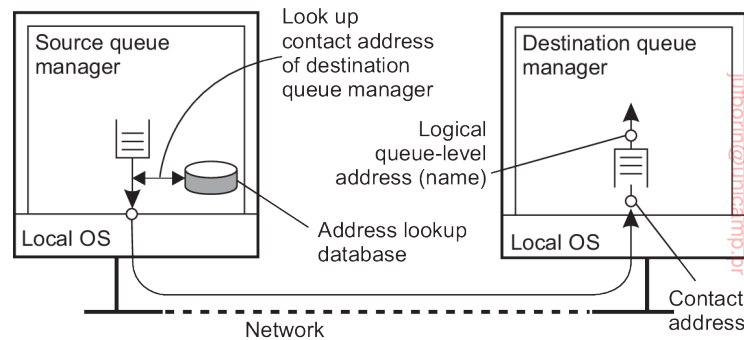


Figure 4.28: The relationship between queue-level naming and network-level addressing.

A second issue that we need to consider is how the name-to-address mapping is actually made available to a queue manager. A common approach is to simply implement the mapping as a lookup table and copy that table to all managers. Obviously, this leads to a maintenance problem, for every time that a new queue is added or named, many, if not all tables need to be updated. There are various ways to alleviate such problems, which we will discuss in Chapter 5.

This brings us to a third issue, related to the problems of efficiently maintaining name-to-address mappings. We have implicitly assumed that if a destination queue at manager QM_B is known to queue manager QM_A , then QM_A can directly contact QM_B to transfer messages. In effect, this means that (the contact address of) each queue manager should be known to all others. Obviously, when dealing with very large message-queuing systems, we will have a scalability problem. In practice, there are often special queue managers that operate as **routers**: they forward incoming messages to other queue managers. In this way, a message-queuing system may gradually grow into a complete, application-level, **overlay network**.

If only a few routers need to know about the network topology, then a source queue manager need only to know to which adjacent router, say R , it should forward a message, given a destination queue. Router R , in turn, may only need to keep track of its adjacent routers to see where to forward the message to, and so on. Of course, we still need to have name-to-address mappings for all queue managers, including the routers, but it is not difficult to imagine that such tables can be much smaller and easier to maintain.

Message brokers

An important application area of message-queuing systems is integrating existing and new applications into a single, coherent distributed information system. If we assume that communication with an application takes place through messages, then integration requires that applications can understand the messages they receive. In practice, this requires the sender to have its outgoing messages in the same format as that of the receiver, but also that its messages adhere to the same semantics as those expected by the receiver. Sender and receiver essentially need to speak the same language, that is, adhere to the same messaging protocol.

The problem with this approach is that each time an application A is added to the system having its own messaging protocol, then for each other application B that is to communicate with A we will need to provide the means for converting their respective messages. In a system with N applications, we will thus need $N \times N$ messaging protocol converters.

An alternative is to agree on a common messaging protocol, as is done with traditional network protocols. Unfortunately, this approach will generally not work for message-queuing systems. The problem is the level of abstraction at which these systems operate. A common messaging protocol makes sense only if the collection of processes that make use of that protocol indeed have enough in common. If the collection of applications that make up a distributed information system is highly diverse (which it often is), then inventing a one-size-fits-all solution is simply not going to work.

If we focus only on the format and meaning of messages, commonality can be achieved by lifting the level of abstraction as is done with XML messages. In this case, messages carry information on their own organization, and what has been standardized is the way that they can describe their content. As a consequence, an application can provide information on the organization of its messages that can be automatically processed. Of course, this information is generally not enough: we also need to make sure that the *semantics* of messages are well understood.

Given these problems, the general approach is to learn to live with differences, and try to provide the means to make conversions as simple as possible. In message-queuing systems, conversions are handled by special nodes in a queuing network, known as **message brokers**. A message broker acts as an application-level gateway in a message-queuing system. Its main purpose is to convert incoming messages so that they can be understood by the destination application. Note that to a message-queuing system, a message broker is just another application, as shown in Figure 4.29. In other words, a message broker is generally not considered to be an integral part of the queuing system.

A message broker can be as simple as a reformatter for messages. For example, assume an incoming message contains a table from a database in which records are separated by a special *end-of-record* delimiter and fields

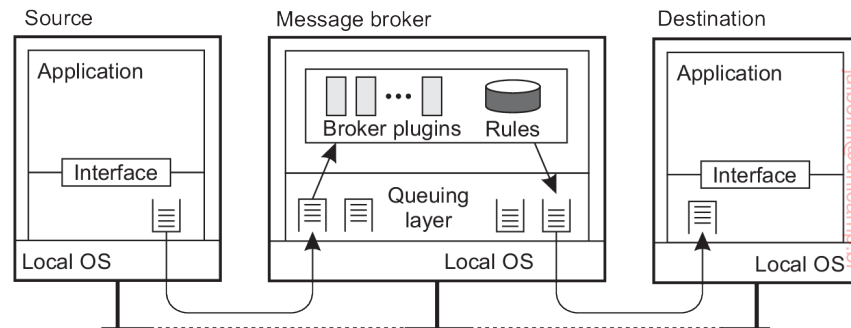


Figure 4.29: The general organization of a message broker in a message-queuing system.

within a record have a known, fixed length. If the destination application expects a different delimiter between records, and also expects that fields have variable lengths, a message broker can be used to convert messages to the format expected by the destination.

In a more advanced setting, a message broker may act as an application-level gateway, in which information on the messaging protocol of several applications has been encoded. In general, for each pair of applications, we will have a separate subprogram capable of converting messages between the two applications. In Figure 4.29, this subprogram is drawn as a **plugin** to emphasize that such parts can be dynamically plugged in, or removed from a broker.

Finally, note that in many cases a message broker is used for advanced enterprise application integration (EAI), as we discussed in Section 1.3. In this case, rather than (only) converting messages, a broker is responsible for *matching* applications based on the messages that are being exchanged. In such a **publish-subscribe model**, applications send messages in the form of *publishing*. In particular, they may publish a message on topic X, which is then sent to the broker. Applications that have stated their interest in messages on topic X, that is, who have *subscribed* to those messages, will then receive these messages from the broker. More advanced forms of mediation are also possible.

At the heart of a message broker lies a repository of rules for transforming a message of one type to another. The problem is defining the rules and developing the plugins. Most message broker products come with sophisticated development tools, but the bottom line is still that the repository needs to be filled by experts. Here we see a perfect example where commercial products are often misleadingly said to provide “intelligence,” where, in fact, the only intelligence is to be found in the heads of those experts.

Note 4.12 (More information: A note on message-queuing systems)

Considering what we have said about message-queuing systems, it would appear that they have long existed in the form of implementations for e-mail services. E-mail systems are generally implemented through a collection of mail servers that store and forward messages on behalf of the users on hosts directly connected to the server. Routing is generally left out, as e-mail systems can make direct use of the underlying transport services. For example, in the mail protocol for the Internet, SMTP [Postel, 1982], a message is transferred by setting up a direct TCP connection to the destination mail server.

What makes e-mail systems special compared to message-queuing systems is that they are primarily aimed at providing direct support for end users. This explains, for example, why a number of groupware applications are based directly on an e-mail system [Khoshafian and Buckiewicz, 1995]. In addition, e-mail systems may have very specific requirements such as automatic message filtering, support for advanced messaging databases (e.g., to easily retrieve previously stored messages), and so on.

General message-queuing systems are not aimed at supporting only end users. An important issue is that they are set up to enable persistent communication between processes, regardless of whether a process is running a user application, handling access to a database, performing computations, and so on. This approach leads to a different set of requirements for message-queuing systems than pure e-mail systems. For example, e-mail systems generally need not provide guaranteed message delivery, message priorities, logging facilities, efficient multicasting, load balancing, fault tolerance, and so on for general usage.

General-purpose message-queuing systems, therefore, have a wide range of applications, including e-mail, workflow, groupware, and batch processing. However, as we have stated before, the most important application area is the integration of a (possibly widely-dispersed) collection of databases and applications into a federated information system [Hohpe and Woolf, 2004]. For example, a query expanding several databases may need to be split into subqueries that are forwarded to individual databases. Message-queuing systems assist by providing the basic means to package each subquery into a message and routing it to the appropriate database. Other communication facilities we have discussed in this chapter are far less appropriate.

Example: IBM's WebSphere message-queuing system

To help understand how message-queuing systems work in practice, let us take a look at the message-queuing system that is part of IBM's WebSphere. Formerly known as MQSeries, it is now referred to as **WebSphere MQ**. There is a wealth of documentation on WebSphere MQ, and in the following we can only resort to the basic principles. Taylor [2012] provides a gentle introduction to WebSphere MQ, whereas further details can be found in [Davies and Broadhurst, 2005; Aranha et al., 2013].

Overview

The basic architecture of an MQ queuing network is quite straightforward, and is shown in Figure 4.30. All queues are managed by **queue managers**. A queue manager is responsible for removing messages from its send queues, and forwarding those to other queue managers. Likewise, a queue manager is responsible for handling incoming messages by picking them up from the underlying network and subsequently storing each message in the appropriate input queue. To give an impression of what messaging can mean: a message has a maximum default size of 4 MB, but this can be increased up to 100 MB. A queue is normally restricted to 2 GB of data, but depending on the underlying operating system, this maximum can be easily set higher.

Queue managers are pairwise connected through **message channels**, which are an abstraction of transport-level connections. A message channel is a uni-directional, reliable connection between a sending and a receiving queue manager, through which queued messages are transported. For example, an Internet-based message channel is implemented as a TCP connection. Each of the two ends of a message channel is managed by a **message channel agent (MCA)**. A sending MCA is basically doing nothing else than checking send queues for a message, wrapping it into a transport-level packet, and sending it along the connection to its associated receiving MCA. Likewise, the basic task of a receiving MCA is listening for an incoming packet, unwrapping it, and subsequently storing the unwrapped message into the appropriate queue.

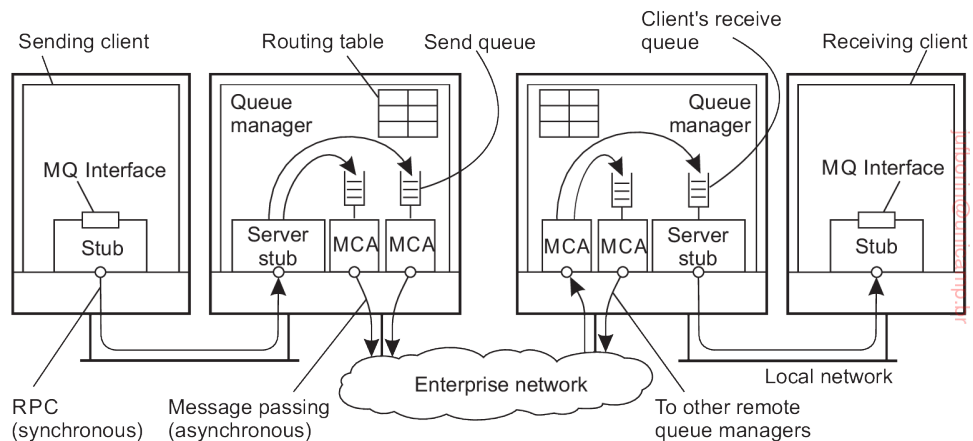


Figure 4.30: General organization of IBM's message-queuing system.

Queue managers can be linked into the same process as the application for which it manages the queues. In that case, the queues are hidden from the application behind a standard interface, but effectively can be directly manipulated by the application. An alternative organization is one in which queue managers and applications run on separate machines. In that case,

the application is offered the same interface as when the queue manager is co-located on the same machine. However, the interface is implemented as a proxy that communicates with the queue manager using traditional RPC-based synchronous communication. In this way, MQ basically retains the model that only queues local to an application can be accessed.

Channels

An important component of MQ is formed by the message channels. Each message channel has exactly one associated send queue from which it fetches the messages it should transfer to the other end. Transfer along the channel can take place only if both its sending and receiving MCA are up and running. Apart from starting both MCAs manually, there are several alternative ways to start a channel, some of which we discuss next.

One alternative is to have an application directly start its end of a channel by activating the sending or receiving MCA. However, from a transparency point of view, this is not a very attractive alternative. A better approach to start a *sending* MCA is to configure the channel's send queue to set off a trigger when a message is first put into the queue. That trigger is associated with a handler to start the sending MCA so that it can remove messages from the send queue.

Another alternative is to start an MCA over the network. In particular, if one side of a channel is already active, it can send a control message requesting that the other MCA is to be started. Such a control message is sent to a daemon listening to a well-known address on the same machine as where the other MCA is to be started.

Channels are stopped automatically after a specified time has expired during which no more messages were dropped into the send queue.

Each MCA has a set of associated attributes that determine the overall behavior of a channel. Some of the attributes are listed in Figure 4.31. Attribute values of the sending and receiving MCA should be compatible and perhaps negotiated first before a channel can be set up. For example, both MCAs should obviously support the same transport protocol. An example of a nonnegotiable attribute is whether or not messages are to be delivered in the same order as they are put into the send queue. If one MCA wants FIFO delivery, the other must comply. An example of a negotiable attribute value is the maximum message length, which will simply be chosen as the minimum value specified by either MCA.

Message transfer

To transfer a message from one queue manager to another (possibly remote) queue manager, it is necessary that each message carries its destination address, for which a transmission header is used. An address in MQ consists of

Attribute	Description
Transport type	Determines the transport protocol to be used
FIFO delivery	Indicates that messages are to be delivered in the order they are sent
Message length	Maximum length of a single message
Setup retry count	Maximum number of retries to start up the remote MCA
Delivery retries	Maximum times MCA will try to put received message into queue

Figure 4.31: Some attributes associated with message channel agents.

two parts. The first part consists of the name of the queue manager to which the message is to be delivered. The second part is the name of the destination queue resorting under that manager to which the message is to be appended.

Besides the destination address, it is also necessary to specify the route that a message should follow. Route specification is done by providing the name of the local send queue to which a message is to be appended. Thus it is not necessary to provide the full route in a message. Recall that each message channel has exactly one send queue. By telling to which send queue a message is to be appended, we effectively specify to which queue manager a message is to be forwarded.

In most cases, routes are explicitly stored inside a queue manager in a routing table. An entry in a routing table is a pair (destQM, sendQ), where destQM is the name of the destination queue manager, and sendQ is the name of the local send queue to which a message for that queue manager should be appended. (A routing table entry is called an alias in MQ.)

It is possible that a message needs to be transferred across multiple queue managers before reaching its destination. Whenever such an intermediate queue manager receives the message, it simply extracts the name of the destination queue manager from the message header, and does a routing-table look-up to find the local send queue to which the message should be appended.

It is important to realize that each queue manager has a systemwide unique name that is effectively used as an identifier for that queue manager. The problem with using these names is that replacing a queue manager, or changing its name, will affect all applications that send messages to it. Problems can be alleviated by using a **local alias** for queue manager names. An alias defined within a queue manager QM₁ is another name for a queue manager QM₂, but which is available only to applications interfacing to QM₁. An alias allows the use of the same (logical) name for a queue, even if the queue manager of that queue changes. Changing the name of a queue manager requires that we change its alias in all queue managers. However, applications can be left unaffected.

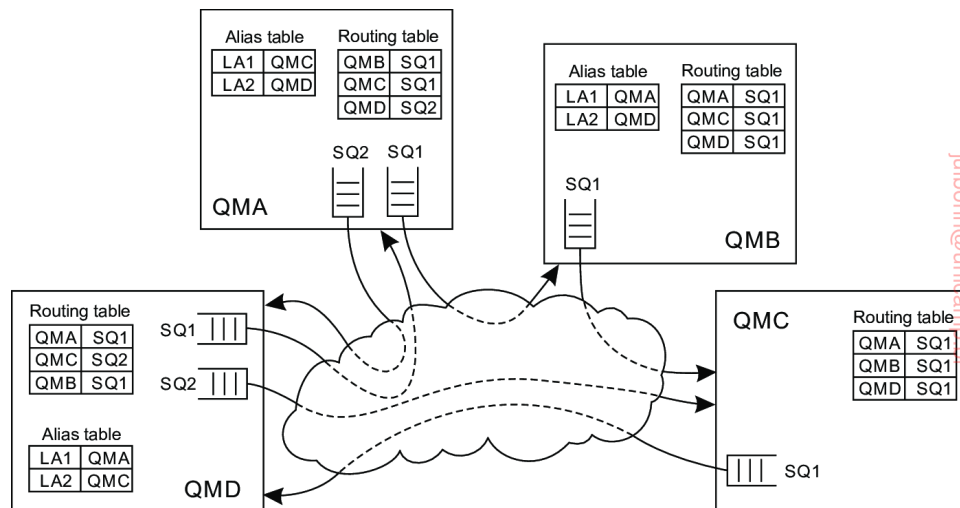


Figure 4.32: The general organization of an MQ queuing network using routing tables and aliases.

The principle of using routing tables and aliases is shown in Figure 4.32. For example, an application linked to queue manager QMA can refer to a remote queue manager using the local alias LA₁. The queue manager will first look up the actual destination in the alias table to find it is queue manager QMC. The route to QMC is found in the routing table, which states that messages for QMC should be appended to the outgoing queue SQ₁, which is used to transfer messages to queue manager QMB. The latter will use its routing table to forward the message to QMC.

Following this approach of routing and aliasing leads to a programming interface that is relatively simple, called the **Message Queue Interface (MQI)**. The most important operations of MQI are summarized in Figure 4.33.

Operation	Description
MQOPEN	Open a (possibly remote) queue
MQCLOSE	Close a queue
MQPUT	Put a message into an opened queue
MQGET	Get a message from a (local) queue

Figure 4.33: Operations available in the message-queuing interface.

To put messages into a queue, an application calls MQOPEN, specifying a destination queue in a specific queue manager. The queue manager can be named using the locally-available alias. Whether the destination queue is actually remote or not is completely transparent to the application. MQOPEN should also be called if the application wants to get messages from its local

queue. Only local queues can be opened for reading incoming messages. When an application is finished with accessing a queue, it should close it by calling MQCLOSE.

Messages can be written to, or read from, a queue using MQPUT and MQGET, respectively. In principle, messages are removed from a queue on a priority basis. Messages with the same priority are removed on a first-in, first-out basis, that is, the longest pending message is removed first. It is also possible to request for specific messages. Furthermore, MQ provides facilities to signal applications when messages have arrived, thus avoiding that an application will continuously have to poll a message queue for incoming messages.

An interesting observation, and one that is common to modern message-queuing systems, is that *persistent* messaging is not obtained by simply making queues persistent. Instead, a message is marked as being persistent, and it is a queue manager's job to see to it that the message can survive a crash. As a consequence, a queue can simultaneously store persistent and nonpersistent messages.

Managing overlay networks

From the description so far, it should be clear that an important part of managing MQ systems is connecting the various queue managers into a consistent overlay network. Moreover, this network needs to be maintained over time. For small networks, this maintenance will not require much more than average administrative work, but matters become complicated when message queuing is used to integrate and disintegrate large existing systems.

A major issue with MQ is that overlay networks need to be manually administrated. This administration not only involves creating channels between queue managers, but also filling in the routing tables. Obviously, this can grow into a nightmare. Unfortunately, management support for MQ systems is advanced only in the sense that an administrator can set virtually every possible attribute, and tweak any thinkable configuration. However, the bottom line is that channels and routing tables need to be manually maintained.

At the heart of overlay management is the **channel control function** component, which logically sits between message channel agents. This component allows an operator to monitor exactly what is going on at two end points of a channel. In addition, it is used to create channels and routing tables, but also to manage the queue managers that host the message channel agents. In a way, this approach to overlay management strongly resembles the management of cluster servers where a single administration server is used. In the latter case, the server essentially offers only a remote shell to each machine in the cluster, along with a few collective operations to handle groups of machines. The good news about distributed-systems management is that it offers lots of opportunities if you are looking for an area to explore new solutions to serious problems.

Example: Advanced Message Queuing Protocol (AMQP)

An interesting observation about message-queuing systems is that they have been developed in part to allow legacy applications to interoperate, yet at the same time we see that when it comes to operations between different message-queuing systems, we often hit a wall. As a consequence, once an organization chooses to use a message-queuing system from manufacturer X, they may have to settle for solutions that only X provides. Message-queuing solutions are thus in large part proprietary solutions. So much for openness.

In 2006, a working group was formed to change this situation, which resulted in the specification of the **Advanced Message-Queuing Protocol**, or simply **AMQP**. There are different versions of AMQP, with version 1.0 being the most recent one. There are also various implementations of AMQP, notably of versions prior to 1.0, which by the time version 1.0 was established, had gained considerable popularity. Because a pre-1.0 version is so different from the 1.0 version, yet has also a steady user base, we may see various pre-1.0 AMQP servers exist next to (their undeniably incompatible) 1.0 servers. So much for openness.

In this section, we will describe AMQP, but will more or less deliberately mix the pre-1.0 and 1.0 versions, sticking to the essentials and spirit of AMQP. Details can be found in the specifications [Group, 2008; OASIS, 2011]. Implementations of AMQP include RabbitMQ [Videla and Williams, 2012] and Apache's Qpid.

Basics

AMQP evolves around applications, queue managers, and queues. Taking an approach that is common for many networking situations, we make a distinction between AMQP as a *messaging service*, the actual *messaging protocol*, and, finally, the *messaging interface* as offered to applications. To this end, it is easiest to consider the situation of having only a single queue manager, running as a single, separate server forming the implementation of AMQP as a service. An application communicates with this queue manager through a local interface. Between an application and the queue manager communication proceeds according to the AMQP protocol.

This situation is shown in Figure 4.34 and should look familiar. The AMQP stub shields the application (as well as the queue manager) from the details concerning message transfer and communication in general. At the same time, it implements a message-queuing interface, allowing the application to make use of AMQP as a message-queuing service. Although the distinction between AMQP stub and queue manager is made explicit for queue managers, the strictness of the separation is left to an implementation. Nevertheless, if not strict, conceptually there is a distinction between handling queues and handling related communication as we shall make clear shortly.

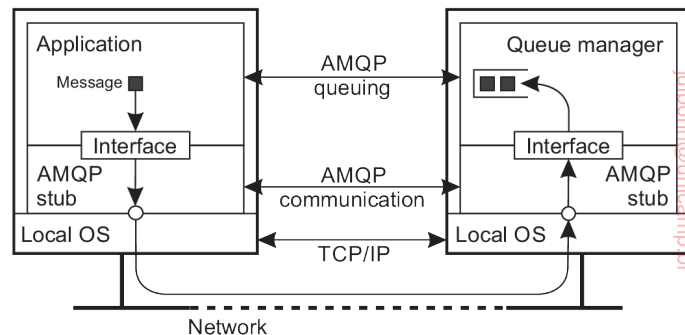


Figure 4.34: An overview of a single-server AMQP instance.

AMQP communication

AMQP allows an application to set up a **connection** to a queue manager; a connection is a container for a number of one-way **channels**. Whereas the lifetime of a channel can be highly dynamic, connections are assumed to be relatively stable. This difference between connection and channel allows for efficient implementations, notably by using a single transport-layer TCP connection to multiplex lots of different channels between an application and a queue manager. In practice, AMQP assumes TCP is used for establishing AMQP connections.

Bidirectional communication is established through **sessions**: a logical grouping of two channels. A connection may have multiple sessions, but note that a channel need not necessarily be part of a session.

Finally, to actually transfer messages, a **link** is needed. Conceptually, a link, or rather its end points, keep track of the status of messages that are being transferred. It thus provides fine-grained flow control between an application and a queue manager, and, indeed, different control policies can be put simultaneously in place for different messages that are transferred through the same session of connection. Flow control is established through credits: a receiver can tell the sender how many messages it is allowed to send over a specific link.

When a message is to be transferred, the application passes it to its local AMQP stub. As mentioned, each message transfer is associated with one specific link. Message transfer normally proceeds in three steps.

1. At the sender's side, the message is assigned a unique identifier and is recorded *locally* to be in an **unsettled state**. The stub subsequently transfers the message to the server, where the AMQP stub also records it as being in an unsettled state. At that point, the server-side stub passes it to the queue manager.

2. The receiving application (in this case the queue manager), is assumed to handle the message and normally reports back to its stub that it is finished. The stub passes this information to the original sender, at which point the message at the original sender's AMQP stub enters a **settled state**.
3. The AMQP stub of the original sender now tells the stub of the original receiver that message transfer has been settled (meaning that the original sender will forget about the message from now on). The receiver's stub can now also discard anything about the message, formally recording it as being settled as well.

Note that because the receiving application can indicate to the underlying AMQP communication layer that it is done with a message, AMQP enables true end-to-end communication reliability. In particular, the application, be it a client application or an actual queue manager, can instruct the AMQP communication layer to keep hold of a message (i.e., a message stays in the unsettled state).

AMQP messaging

Messaging in AMQP logically takes place at the layer above the one handling communication. It is here that an application can indicate what needs to be done with a message, but can also see what has happened so far. Messaging formally takes place between two **nodes**, of which there are three types: a producer, a consumer, or a queue. Typically, producer and consumer nodes represent regular applications, whereas queues are used to store and forward messages. Indeed, a queue manager will typically consist of multiple queue nodes. In order for message transfer to take place, two nodes will have to establish a link between them.

The receiver can indicate to the sender whether its message was accepted (meaning that it was successfully processed), or rejected. Note that this means that a notification is returned to the original sender. AMQP also supports fragmentation and assembly of large messages for which additional notifications are sent.

Of course, an important aspect of AMQP is its support for *persistent* messaging. Achieving persistence is handled through several mechanisms. First and foremost important, a message can be marked as durable, indicating that the source expects any intermediate node, such as a queue, to be able to recover in the case of a failure. An intermediate node that cannot guarantee such durability will have to reject a message. Second, a source or target node can also indicate its durability: if durable, will it maintain its state, or will it also maintain the (unsettled) state of durable messages? Combining the latter with durable messages effectively establishes reliable message transfer and persistent messaging.

AMQP is truly a messaging protocol in the sense that it does not by itself support, for example, publish-subscribe primitives. It expects that such issues are handled by more advanced, proprietary queue managers, akin to the message brokers discussed in Section 4.3.

Finally, there is no reason why a queue manager cannot be connected to another queue manager. In fact, it is quite common to organize queue managers into an overlay network in which messages are routed from producers to their consumers. AMQP does not specify how the overlay network should be constructed and managed, and, indeed, different providers of AMQP-based systems offer different solutions. Of particular importance is specifying how messages should be routed through the network. The bottom line is that administrators will need to do a lot of this specification manually. Only in cases where overlays have regular structures, such as cycles or trees, it becomes easier to provide the necessary routing details.

4.4 Multicast communication

An important topic in communication in distributed systems is the support for sending data to multiple receivers, also known as multicast communication. For many years, this topic has belonged to the domain of network protocols, where numerous proposals for network-level and transport-level solutions have been implemented and evaluated [Janic, 2005; Obraczka, 1998]. A major issue in all solutions was setting up the communication paths for information dissemination. In practice, this involved a huge management effort, in many cases requiring human intervention. In addition, as long as there is no convergence of proposals, ISPs have shown to be reluctant to support multicasting [Diot et al., 2000].

With the advent of peer-to-peer technology, and notably structured overlay management, it became easier to set up communication paths. As peer-to-peer solutions are typically deployed at the application layer, various application-level multicasting techniques have been introduced. In this section, we will take a brief look at these techniques.

Multicast communication can also be accomplished in other ways than setting up explicit communication paths. As we also explore in this section, gossip-based information dissemination provides simple (yet often less efficient) ways for multicasting.

Application-level tree-based multicasting

The basic idea in application-level multicasting is that nodes organize into an overlay network, which is then used to disseminate information to its members. An important observation is that network routers are not involved in group membership. As a consequence, the connections between nodes in the overlay network may cross several physical links, and as such, routing